Get unlimited access to the best of Medium for less than $1/week. Become a member

Expedia Group Technology

EXPEDIA GROUP TECHNOLOGY — DATA

Apache Spark Structured Streaming — Output Sinks (3 of 6)

Straight Outta Spark Streaming



Neeraj Bhadani · Follow

Published in Expedia Group Technology · 7 min read · Feb 18, 2021

96 2



Photo by Matthew Feeney on Unsplash

In previous posts of this series, we discussed an overview as well as input sources for Apache Spark™ Streaming. This time, we discuss the available range of **output sinks**.

You may also be interested in my other posts on Apache Spark.

- Apache Spark Structured Streaming — First Streaming Example
- Apache Spark Structured Streaming — Input Sources
- Deep Dive into Apache Spark DateTime Functions
- Working with JSON in Apache Spark
- Deep Dive into Apache Spark Window Functions
- Deep Dive into Apache Spark Array Functions
- Start Your Journey with Apache Spark

Output Sinks

In Spark Streaming, output sinks store results into external storage.

- **Console sink**: Displays the content of the DataFrame to `console`. In this series, we have only used console sink, refer to previous posts for details.
- **File sink**: Stores the contents of a DataFrame in a file within a directory. Supported file formats are `csv`, `json`, `orc`, and `parquet`.
- **Kafka sink**: Publishes data to a Kafka® topic.
- **Foreach sink**: Applies to each row of a DataFrame and can be used when writing custom logic to store data.
- **ForeachBatch sink**: Applies to each micro-batch of a DataFrame and also can be used when writing custom logic to store data.

It's time to get our hands dirty; let's discuss each output sink in detail.

Setup

We use the same `csv` data files as in the input sources post of this series to create a streaming DataFrame from a file source. Please refer to this blog for more details on the code below.

```scala
// Import libraries
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._
import org.apache.spark.sql.Column
import org.apache.spark.sql.types._

// Create Spark Session
val spark = SparkSession
  .builder()
  .master("local")
  .appName("File Sink")
  .getOrCreate()

// Set Spark logging level to ERROR.
spark.sparkContext.setLogLevel("ERROR")

// Define Schema
val schema = StructType(List(
  StructField("Date", StringType, true),
  StructField("Open", DoubleType, true),
  StructField("High", DoubleType, true),
  StructField("Low", DoubleType, true),
  StructField("Close", DoubleType, true),
  StructField("Adjusted Close", DoubleType, true),
  StructField("Volume", DoubleType, true)
))

// Extract the Name of the stock from the file name.
def getFileName : Column = {
  val file_name = reverse(split(input_file_name(), "/")).getItem(0)
  split(file_name, "_").getItem(0)
}

// Create Streaming DataFrame by reading data from socket.
val initDF = spark
  .readStream
  .option("maxFilesPerTrigger", 2)
  .option("header", true)
  .schema(schema)
  .csv("data/stream")
  .withColumn("Name", getFileName)
```

ss_sink_setup.scala hosted with ❤ by GitHub view raw

Output sink — Console

We covered the `console` output sink multiple times in a previous post.

Output sink — File

The file sink stores the contents of a streaming DataFrame to a specified directory and format. We use `initDf` (created above) and apply a simple transformation before storing it to the file system.

Transformation

Perform a very simple transformation by selecting a few columns from `initDF`

```
val resultDf = initDF.select("Name", "Date", "Open", "Close")
```

Output to file sink

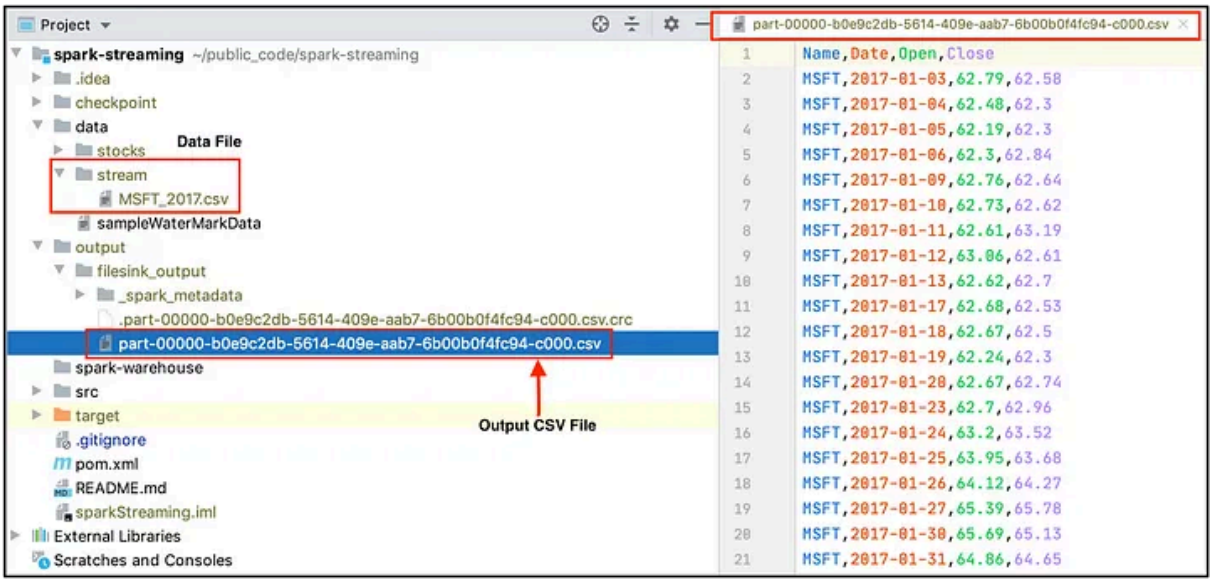
Note: File sink only supports `append` output mode.

Store the content of DataFrame `resultDf` using file sink.

```
resultDf
  .writeStream
  .outputMode("append") // Filesink only support Append mode.
  .format("csv") // supports these formats : csv, json, orc, parquet
  .option("path", "output/filesink_output")
  .option("header", true)
  .option("checkpointLocation", "checkpoint/filesink_checkpoint")
  .start()
  .awaitTermination()
```

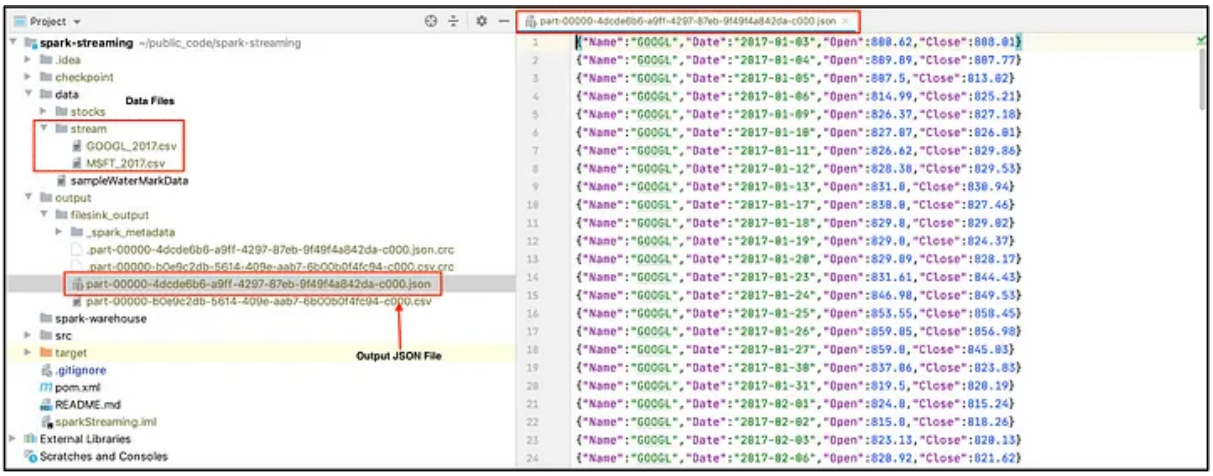
ss_sink_file.scala hosted with ❤ by GitHub view raw

We store the results into `csv` files. We also use checkpoint, which we discuss in a separate blog post. Execute the code above and check the output folder `output/filesink_output .`



The output of our `resultDf` DataFrame is stored in directory `output/filesink_output` . Check out the selected columns (`Name` , `Date` , `Open` , `Close`) in the `csv` on the right side of the snapshot.

Now change the file format to `json` and review the output. The content of our `resultDf` DataFrame is stored in a `json` file.



You can try `orc` and `parquet` formats as well. For easy reference, you can find the complete code on ***GitHub***.

Output sink — Kafka

With the `kafka` sink, we publish the content of our streaming DataFrame to a Kafka topic using the `initDf` DataFrame defined in the Setup section above. You can refer to this to install Kafka and create a Kafka topic.

Transformation

Kafka expects data in a column name `value`, so let's concatenate all the column values and store them in one `value` column.

```
val resultDf = initDf.withColumn("value",
                                 concat_ws("|",$"Name",$"Date",$"High",$"Low",$"Open",$"Close"))
```

ss_sink_kafka_tranform.scala hosted with ❤ by GitHub view raw

Output to Kafka sink

```
resultDf
  .writeStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .option("topic", "testConsumer")
  .option("checkpointLocation", "checkpoint/kafka_checkpoint")
  .start()
  .awaitTermination()
```

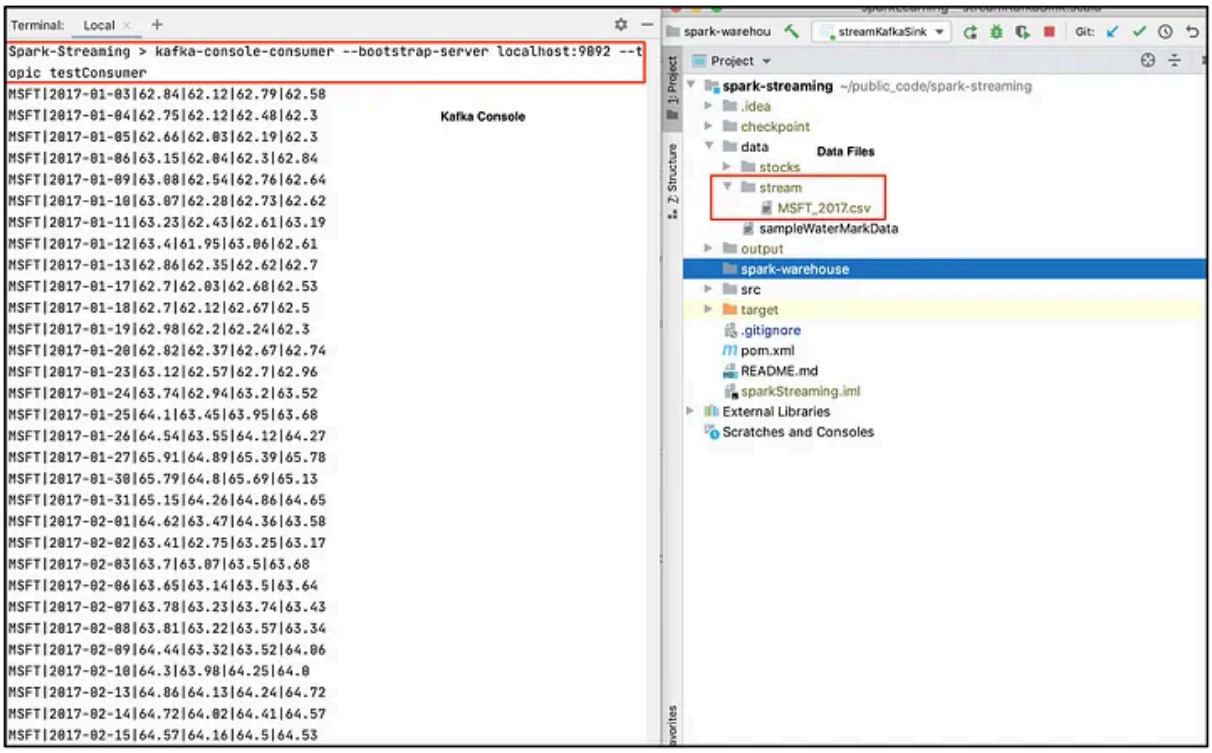
ss_sink_kafka.scala hosted with ❤ by GitHub view raw

We pass the value `"kafka"` to `format()` and specify other necessary Kafka information like `"kafka.bootstrap.servers"` and the `"topic"` to which we publish.

Execute our streaming application

We use the command below to consume the data which our streaming application published to Kafka.

```
kafka-console-consumer --bootstrap-server localhost:9092 --topic
testConsumer
```



In the screenshot above, as soon as we drop a file into `data/stream` (a folder in the right window), we see our transformed output on Kafka's console on the left window (which is pipe `|` separated). For easy reference, you can find the complete code on ***GitHub***.

Output sink — foreachBatch

So far we discussed sinks where the output system was already defined like `file`, `kafka`, or `console`. What if we would like to store data in any arbitrary storage like a NoSQL DB (for example MongoDB) or a Relational DB (like MySQL). By using `foreach` and `foreachBatch` we can write custom logic to store data. `foreach` performs custom write logic on each row and `foreachBatch` performs custom write logic on each micro-batch.

Using `foreachBatch`, we write each micro-batch to storage defined in our custom logic. In this case, we store the output of our streaming application to MySQL DB.

Transformation

Select a few columns from `initDF`

```scala
val resultDf = initDF.select("Name", "Date", "open")
```

ss_sink_foreachBatch_transform.scala hosted with ❤ by GitHub view raw

Output to foreachBatch sink

`foreachBatch` takes a function that expects 2 parameters, first: micro-batch as DataFrame or Dataset and second: unique id for each batch. First, create a function with custom write logic to save a micro-batch to MySQL.

```scala
def saveToMySql = (df: Dataset[Row], batchId: Long) => {
    val url = """jdbc:mysql://localhost:3306/training"""

    df
        .withColumn("batchId", lit(batchId))
        .write.format("jdbc")
        .option("url", url)
        .option("dbtable", "test")
        .option("user", "root")
        .option("password", <mySQL_PASSWORD>)
        .mode("append")
        .save()
}
```

ss_sink_foreachBatch_saveToMySql.scala hosted with ❤ by GitHub view raw

We specified all the necessary details of MySQL DB like URL, DB name (`training`), Table name (`test`), User name (`root`), and Password.

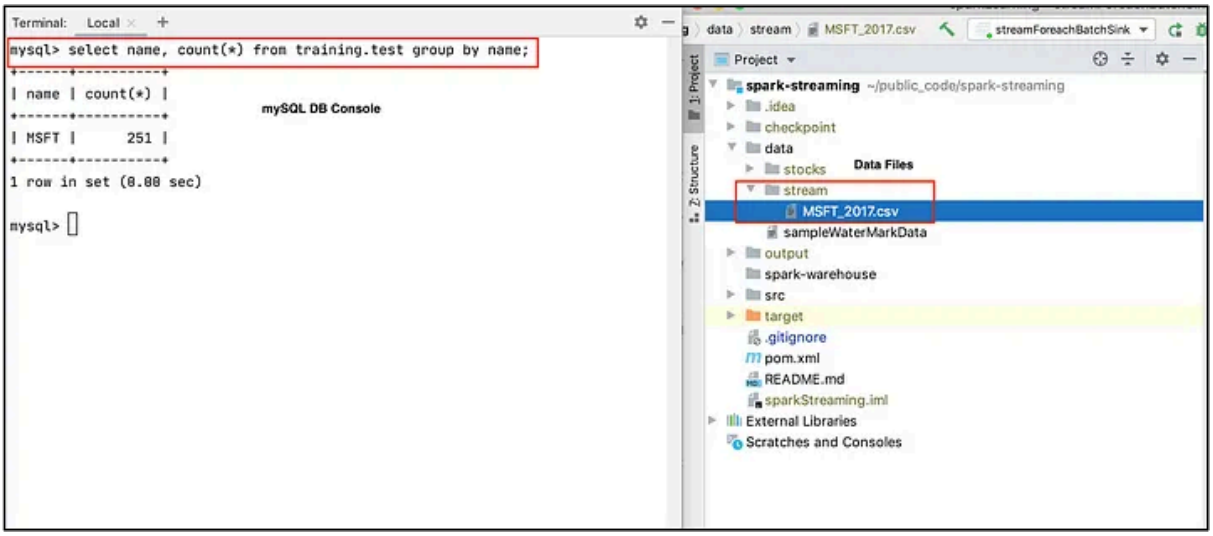
Use the `savetoMySQL()` function to save our streaming DataFrame to MySQL.

```scala
resultDf
  .writeStream
  .outputMode("append")
  .foreachBatch(saveToMySql)
  .start()
  .awaitTermination()
```

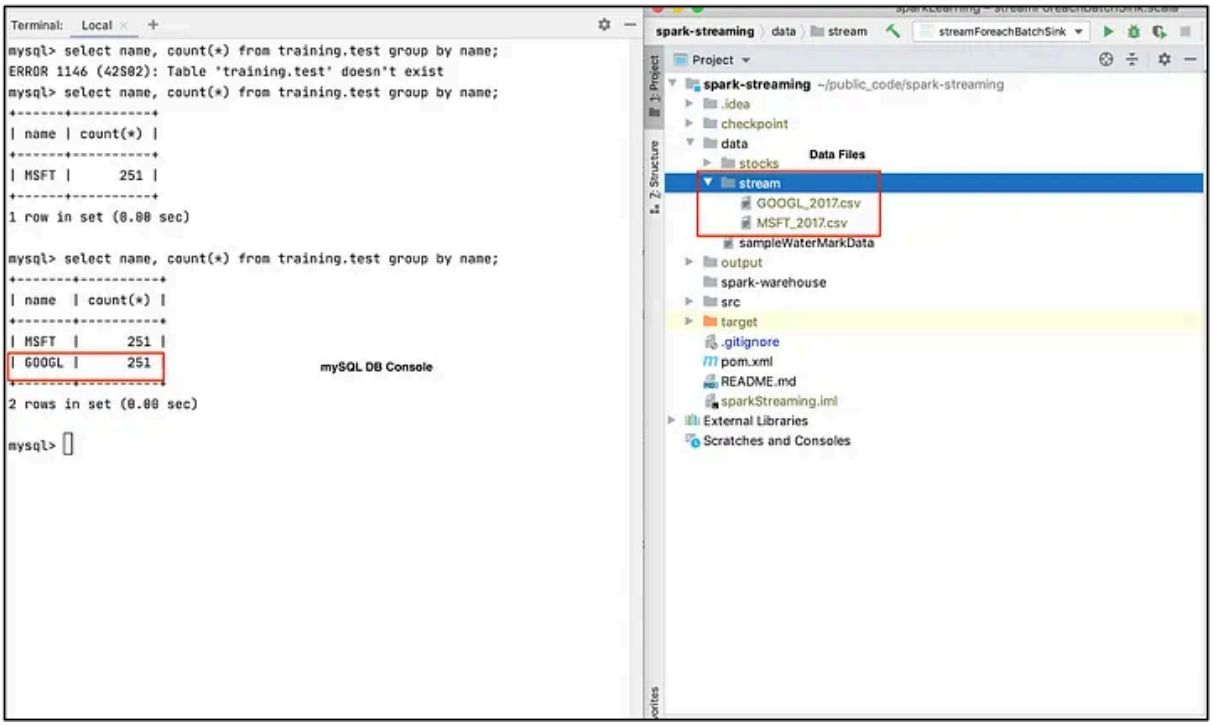
ss_sink_foreachBatch.scala hosted with ❤ by GitHub view raw

We use the `foreachBatch` output sink and pass the `saveToMySql()` function to store data in MySQL. Start the application, ingest the data, and check the

result on MySQL.



In the screenshot above, we dropped the `MSFT_2017.csv` file into `data/stream` on the right window and see the number of records for Microsoft™ in `training.test` MySQL table on the left window.



Here we drop the `GOOGL_2017.csv` file into `data/stream` on the right window and see the number of records for Google in `training.test` MySQL table on

the left window. For easy reference, you can find the complete code on *GitHub*.

Output sink — foreach

Open in app ↗

Medium Search Write A

`processing` mode or if a batch data writer does not exist, then we can use the `foreach` sink for custom write logic.

To use `foreach` we need to implement 3 methods (`open`, `process`, and `close`).

- **open**: function to open connection
- **process**: write data to the specified connection
- **close**: function to close connection

Output to foreach sink

We use `initDF` to read data from a File source. Then we create an instance of the `ForeachWriter` class and implement the `open()`, `process()`, and `close()` methods. We simply print records to console, so we use `println()` statements in the `process()` function.

```
1   val customWriter = new ForeachWriter[Row] {
2
3     override def open(partitionId: Long, version: Long) = true
4
5     override def process(value: Row) = {
6       println("Name : "  + value.getAs("Name"))
7       println("Open : "  + value.getAs("Open"))
8       println("Size : " + value.size)
9       println("Values as Seq : " + value.toSeq)
10    }
11
12    override def close(errorOrNull: Throwable) = {}
13  }
```

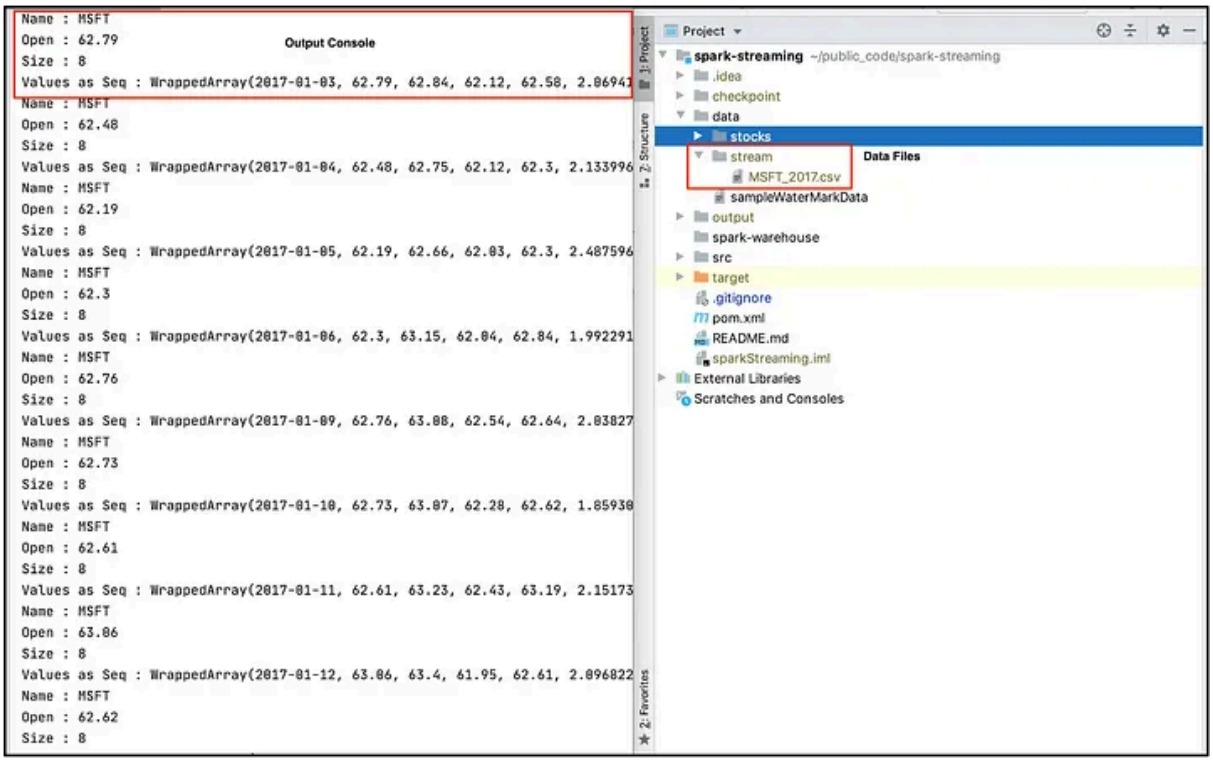
ss_sink_foreach_customWriter.scala hosted with ❤ by GitHub view raw

Use the `ForeachWriter` instance defined above to write data using the `foreach` sink.

```
initDF
  .writeStream
  .outputMode("append")
  .foreach(customWriter)
  .start()
  .awaitTermination()
```

ss_sink_foreach.scala hosted with ❤ by GitHub view raw

We used the `foreach` output sink and provide an instance of `customWriter` defined above. Execute the streaming application and check the result.



As soon as we drop the file `MSFT_2017.csv` into folder `data/stream` on the right-hand window, we see output on the left window defined by `process` method in the `foreach` output sink. For easy reference, you can find the complete code on ***GitHub***.

The next post in this series covers Checkpoints and Triggers.

Here are other blogs on Apache Spark Structured Streaming series.

- Apache Spark Structured Streaming — First Streaming Example
- Apache Spark Structured Streaming — Input Sources
- Apache Spark Structured Streaming — Checkpoints and Triggers
- Apache Spark Structured Streaming — Operations
- Apache Spark Structured Streaming — Watermarking



I hope you enjoyed learning about output sinks used to output data from Spark Streaming!

References

Structured Streaming Programming Guide

Structured Streaming is a scalable and fault-tolerant stream processing engine built on the Spark SQL engine. You can...

spark.apache.org

This blog post is provided for educational purposes only. Certain scenarios may utilize fictitious names and test data for illustrative purposes. The content is intended as a contribution to the technological community at large and may not necessarily reflect any technology currently being implemented or planned for implementation at Expedia Group, Inc.

Software Engineering | Streaming | Data Science | Programming | Technology



Published in Expedia Group Technology

5K Followers · Last published Feb 25, 2025

Follow

Stories from the Expedia Group Technology teams



Written by Neeraj Bhadani

695 Followers · 25 Following

Follow

Data Science and cloud computing enthusiast

Responses (2)

A Akhedkar

What are your thoughts?



Fabrizzio Chávez Tejada
Aug 9, 2024

Hello, is it posible to use checkpoints with foreachBatch output? Thanks.

Reply



YZ
May 21, 2021

Hi Neeraj, thank you for your article firstly.

I encounted one problem when run the streamForeachBatchSink.scala.

The error:

Exception in thread "main" org.apache.spark.sql.streaming.StreamingQueryException: Column "batchId" not found in schema... more

Reply

More from Neeraj Bhadani and Expedia Group Technology

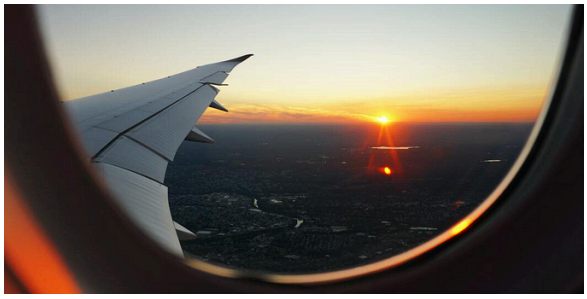


In Expedia Group Technology by Neeraj Bhadani

Deep dive into Apache Spark Window Functions

Window functions operate on groups of data and return values for each record or group

Jun 25, 2020 557 4



In Expedia Group Technology by Ritesh Kapoor

Onion Architecture

Let's slice it like a Pro

Jul 12, 2022 319 8



In Expedia Group Technolo... by Christine Mathies...

A Short Introduction to Apache Iceberg

The data lake team at Expedia Group starts working with table formats, adds Hive...



In Expedia Group Technology by Neeraj Bhadani

Deep Dive into Apache Spark Array Functions

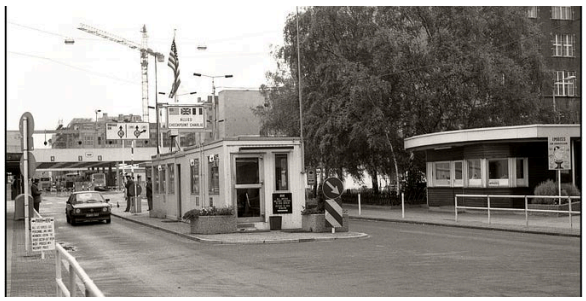
A practical guide to using array functions

Jan 26, 2021 662 3 Jul 16, 2020 201 3

See all from Neeraj Bhadani

See all from Expedia Group Technology

Recommended from Medium

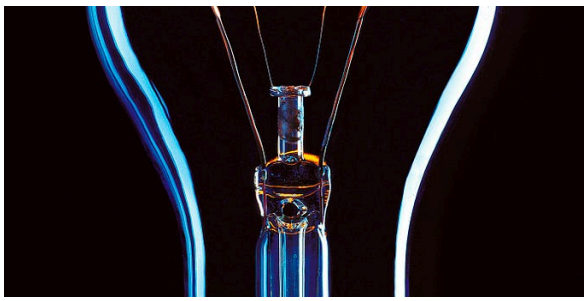


Alon Nisser

Spark structured streaming — checkpointing

Where I explain what I've learned about spark streaming checkpointing, how it's...

Nov 22, 2024 2

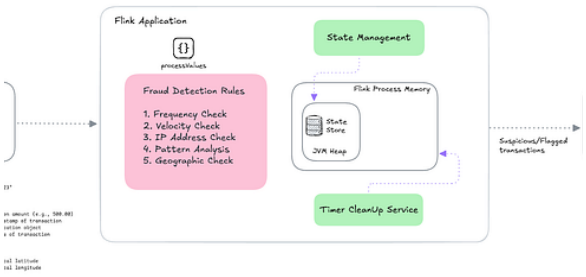


In Towards Data Engineering by Ritam Mukherjee

Building Real-Time Recommendations with Spark,...

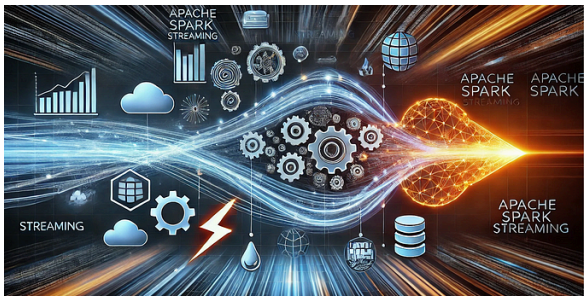
Ever wondered how online stores know exactly what you'd like to buy next

Nov 30, 2024 80



In Yugen.ai Technology Blog by Yugen.ai

Real Time Fraud Detection Using Apache Flink — Part 1



Ashutosh Kumar

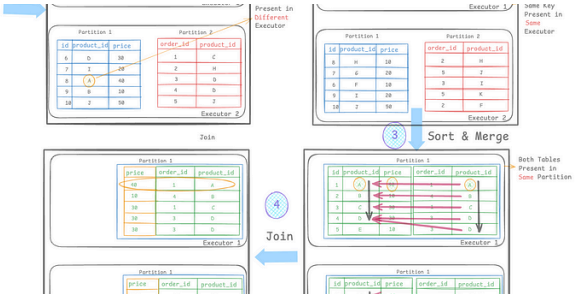
Apache Spark Streaming — All you need to know...

A simple Flink application to detect suspicious transactions

Feb 6 29 1

If you are working in the Analytics, Machine Learning or Data Engineering space, you...

Jan 2 6

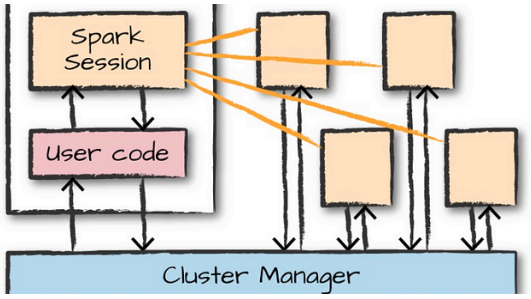


Shantanu Tripathi

Sort Merge Join: Visual Flow inside Executors

Learn the default join strategy used by Spark through visuals.

Dec 29, 2024 60



In Towards Dev by Prem Vishnoi(cloudvala)

Apache Spark Architecture :A Deep Dive into Big Data Processing

Agenda

Feb 6 90 1

See more recommendations